

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1510

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I ____ Shaik. Abdul Razak (192524358)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ____ Shaik. Abdul Razak _____

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy,

- encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
 5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. Website, Mobile Interaction and Customer Transaction Platform

Reverse Engineering Analysis

The platform collects billions of website clicks, mobile interactions, and customer transactions. Such a workload requires a distributed and highly scalable big data architecture.

1. Distributed Storage System

The probable storage layer would use **Hadoop Distributed File System (HDFS)** or cloud object storage such as Amazon S3. HDFS can distribute large datasets across multiple nodes and provide fault tolerance through data replication.

Structured transaction data may be stored in relational databases or data warehouses, while semi-structured clickstream and application logs may be stored in a data lake using formats such as JSON, Avro, or Parquet.

2. Data Ingestion

Apache Kafka is a likely ingestion framework because it can receive billions of events from websites, mobile applications, and transaction systems. Kafka topics can separate different event types such as clicks, payments, and mobile activity.

3. Batch Processing

Apache Spark is likely used for large-scale batch processing. It can clean, transform, aggregate, and analyze historical customer data.

4. Stream Processing

For real-time analytics, **Apache Flink**, Spark Structured Streaming, or Kafka Streams could be used. These engines process incoming events continuously and support real-time reporting.

5. Data Separation and Indexing

Structured transaction data can be stored in relational databases or analytical warehouses. Semi-structured clickstream data can be stored in a data lake.

Data can be partitioned by **date, region, customer, or event type**. Columnar formats such as Parquet improve analytical query performance. Search and indexing technologies such as Elasticsearch can support fast event searches.

6. Scalability and Partitioning

The system would use horizontal scaling by adding additional processing and storage nodes. Kafka partitions can distribute incoming events across brokers. Spark/Flink partitions can distribute processing across worker nodes.

Replication provides fault tolerance, while load balancing prevents individual nodes from becoming bottlenecks.

7. Dashboards and Business Intelligence

Processed data can be loaded into a data warehouse or analytical database. Tools such as **Power BI, Tableau, or Grafana** can connect to this analytics layer to provide dashboards containing sales, customer activity, traffic, and performance KPIs.

Conclusion

The probable architecture combines Kafka, HDFS/cloud storage, Spark, Flink, analytical databases, and BI tools to support massive-scale data processing and real-time analytics.

2. E-Commerce Personalization Platform

Reverse Engineering Analysis

The platform tracks product views, cart additions, abandoned checkouts, and payment behavior. This indicates an event-driven architecture designed for real-time personalization.

1. Event Streaming

Apache Kafka is likely used to collect events from websites, mobile applications, and payment systems. Separate Kafka topics can store events such as product views, cart updates, and payments.

2. Customer Profiling Database

A distributed NoSQL database such as **Cassandra, MongoDB, or DynamoDB** could store customer profiles and frequently changing behavioral information.

A fast caching layer such as **Redis** can store customer sessions, recently viewed products, shopping carts, and recommendation results.

3. Recommendation Workflow

The likely workflow is:

User Event → Kafka → Stream Processing → Customer Profile Update → Recommendation Model → Cache → Personalized Application

Machine learning models can use collaborative filtering, content-based filtering, or hybrid recommendation techniques.

4. Session and Transaction Synchronization

Session IDs and customer IDs can connect clickstream events with transaction records. Streaming processors can combine current browsing behavior with historical customer information to generate real-time recommendations.

5. Distributed Query Processing

Apache Spark SQL, Trino, or Presto can process large datasets for analytical queries.

6. Combining Structured and Semi-Structured Data

Structured order and payment information can be stored in relational databases or warehouses. Semi-structured browsing logs can be stored in a data lake.

Both datasets can be merged using common identifiers such as **Customer ID, Session ID, Product ID, and Order ID.**

7. Scalability and Fault Tolerance

Kafka partitions distribute events across multiple brokers. Processing nodes can be scaled horizontally. Database replication and automatic failover protect customer information.

Caching with Redis reduces database load and provides low-latency access to frequently requested information.

8. Dashboard and KPI Monitoring

Processed data can be connected to Power BI, Tableau, or Grafana. Dashboards can show:

- Conversion rate
- Cart abandonment rate
- Sales revenue
- Average order value
- Product views
- Recommendation performance
- Customer activity

Conclusion

The probable solution uses Kafka, Redis, NoSQL databases, Spark/Flink, distributed query engines, and machine learning to provide scalable real-time e-commerce personalization.

3. Big Data Healthcare System

Reverse Engineering Analysis

The healthcare platform processes patient histories, diagnostic images, wearable sensor streams, and prescriptions. This requires a hybrid architecture because healthcare data contains structured, semi-structured, and unstructured information.

1. Hybrid Database Architecture

Structured information such as patient details, prescriptions, and laboratory results can be stored in relational databases.

Large medical images such as X-rays, CT scans, and MRI images can be stored in object storage or a healthcare data lake.

NoSQL databases can store high-volume wearable sensor data.

2. Data Integration

Hospitals, laboratories, and insurance companies can exchange information using integration tools such as **Apache NiFi** or cloud ETL services.

Healthcare interoperability standards such as **HL7 and FHIR** can be used to exchange patient information between different systems.

3. Streaming Framework

Wearable sensors generate continuous data. **Apache Kafka** can collect sensor events, while **Apache Flink or Spark Structured Streaming** can process them in real time.

For example, abnormal heart rate readings can generate immediate alerts.

4. Machine Learning

Machine learning frameworks such as **TensorFlow, PyTorch, and Spark MLlib** can be used for:

- Disease prediction
- Patient risk scoring
- Medical image analysis
- Readmission prediction
- Patient monitoring

5. Privacy and Security

Healthcare information requires strong protection. The system would likely use:

- Encryption at rest and in transit
- Role-Based Access Control
- Multi-factor authentication
- Data masking and anonymization
- Audit logs
- Secure APIs

Access to sensitive patient information should be limited according to user roles.

6. Analytics Pipeline

The likely pipeline is:

Healthcare Sources → Data Integration → Secure Data Lake → Processing → Machine Learning → Clinical Analytics → Decision Support

Historical data can be processed using Spark for population-level studies, while streaming data can support real-time patient monitoring.

Conclusion

The probable healthcare architecture combines relational databases, NoSQL storage, data lakes, Kafka, Spark/Flink, healthcare interoperability standards, and machine learning while enforcing strict privacy and security controls.

4. Smart City Platform

Reverse Engineering Analysis

The smart city platform continuously collects traffic, CCTV metadata, weather, and public transport information. Such continuous data generation indicates the use of IoT, edge computing, streaming, and geospatial technologies.

1. Edge Computing

Traffic sensors and cameras can connect to edge devices located close to the data sources. Edge computing can filter, aggregate, and analyze data before sending it to the central cloud.

For example, CCTV systems can extract metadata such as vehicle counts rather than continuously transferring complete video streams.

2. Distributed Messaging

Apache Kafka or MQTT can transport sensor and IoT events to the central processing platform.

Kafka provides high-throughput event streaming and allows multiple consumers to process the same data.

3. Centralized Storage

A cloud data lake or HDFS can store historical traffic, weather, transportation, and sensor data.

A time-series database such as **InfluxDB or TimescaleDB** can store continuously changing sensor measurements.

4. Geospatial Analytics

Location-aware databases such as **PostGIS** can store coordinates and geographic information.

Geospatial processing can identify:

- Traffic congestion areas
- Accident locations
- Public transport routes

- Traffic patterns
- Weather-affected regions

5. Streaming Computation

Apache Flink or Spark Structured Streaming can process traffic and transportation events in real time.

Machine learning models can use historical traffic patterns, weather, time, and vehicle density to predict congestion.

6. Batch and Real-Time Analytics

Real-time processing can support immediate traffic management, while batch processing can analyze historical data for long-term urban planning.

For example:

Real-time analytics → Traffic signal adjustment

Batch analytics → Road planning and infrastructure development

7. Security

The platform would likely use:

- Encryption
- Authentication
- Role-Based Access Control
- Network segmentation
- Secure APIs
- Audit logging
- Device authentication

Citizen-related information should be anonymized wherever possible.

8. Visualization

Grafana, Power BI, Tableau, or specialized GIS dashboards can display live traffic maps, congestion levels, weather conditions, and public transportation status.

Conclusion

The smart city architecture probably combines edge computing, Kafka/MQTT, cloud storage, time-series databases, PostGIS, Spark/Flink, machine learning, and visualization tools to provide real-time urban intelligence.

5. Healthcare Analytics System

Reverse Engineering Analysis

The system integrates patient records, wearable devices, laboratory reports, and appointment histories. Because the data is sensitive and comes from multiple sources, a hybrid storage and secure integration architecture is likely.

1. Hybrid Storage Architecture

Structured patient and appointment information can be stored in relational databases.

Laboratory reports and medical documents can be stored in a secure data lake or document database. Wearable readings can be stored in NoSQL or time-series databases.

2. ETL and Interoperability

ETL pipelines can extract data from hospitals and clinics, transform it into a common format, and load it into the centralized analytics platform.

Healthcare interoperability standards such as **HL7** and **FHIR** can allow different healthcare systems to exchange patient information consistently.

Apache NiFi, cloud ETL services, or similar integration platforms can automate the data movement.

3. Real-Time Monitoring

Wearable devices can continuously send health measurements through an IoT gateway. Kafka can stream these measurements to real-time processing systems such as Flink or Spark Structured Streaming.

Abnormal readings can generate alerts for healthcare professionals.

4. Distributed Processing

Apache Spark can process large historical datasets across multiple nodes. It can analyze population-level information to identify trends, disease patterns, and risk factors.

5. Machine Learning

Machine learning models can support:

- Diagnosis risk scoring
- Disease prediction
- Patient deterioration prediction
- Treatment recommendation
- Readmission prediction
- Personalized healthcare

Classification models can estimate disease risk, while regression models can predict continuous health outcomes.

6. Encryption and Compliance

Sensitive healthcare information should be encrypted both during transmission and while stored.

The platform would likely implement:

- Role-Based Access Control
- Multi-factor authentication
- Encryption at rest
- Encryption in transit
- Data anonymization
- Audit trails
- Access monitoring
- Data retention policies

Only authorized healthcare personnel should access sensitive patient information.

7. Analytics Pipeline

The probable pipeline is:

Patient/Hospital Data → ETL & Integration → Secure Storage → Distributed Processing → Machine Learning → Risk Analysis → Clinical Decision Support

Conclusion

The healthcare analytics system likely combines hybrid databases, secure ETL pipelines, HL7/FHIR interoperability, Kafka, Spark/Flink, machine learning, encryption, and auditing to provide scalable healthcare analytics while protecting patient information.

Overall Conclusion

Reverse engineering these systems shows that modern big data platforms commonly combine **distributed storage, event-streaming systems, batch and stream processing, NoSQL databases, cloud data lakes, machine learning, security mechanisms, and visualization tools**. The exact technologies may vary, but the design decisions are generally driven by data volume, velocity, variety, scalability, latency, fault tolerance, privacy, and analytical requirements.